

Entry points: every way into a running system

A plain-language guide to a body of data Klauro produces, written so you can decide how and where to present it. It describes what the information *is* and the shape it comes in — the presentation is yours to design.

Audience: design

Scope: one deployable at a time

Tests: out of scope

Numbers: from a live analysis

START HERE

What we're describing

Klauro analyzes a codebase and, among many things, finds every **entry point** — every place where something *outside* a running program can reach in and make it do work. One quick way to picture it: if a deployable (one independently running piece of software — an API server, a web app, a background worker) is a building, the entry points are the doors. That's the last we'll lean on the metaphor — from here we just call them **entry points**.

A system typically has more entry points than people expect, and they're not all the same *kind*. Some you knock on and wait for an answer. Some are mail slots — you drop something off and walk away. Some trip on their own when the environment changes. The job of this information is to make that full set **legible at a glance** — how many, of what kinds, how they're reached, whether they're protected, and what each one sets in motion.

Who looks at this, and why

An engineer or architect new to a system asking *"how is this thing reached — what's its usage surface?"* · A security reviewer asking *"what's exposed to the public versus protected behind a login?"* · Anyone gauging integration points or blast radius before making a change. The through-line: **the surface area of contact between this system and the outside world.**

One thing to hold onto: this is always **per deployable**. A web app's entry points are pages and button clicks; an API server's are tool calls and messages. The same system can

contain both, and they look nothing alike. The viewer is always looking at *one* deployable, with the ability to switch to another.

THE MENTAL MODEL

Three families of entry point

This is the most important structure in the dataset. Every entry point falls into one of three families based on *how the interaction behaves* — a split Klauro already computes. The counts are from a real analysis, tests excluded.

Request & wait SYNCHRONOUS

The front counter — someone asks, and waits for the answer.

A caller sends a request and holds the line until the system responds. The most familiar kind, and usually the one people mean by "the API."

495 UI interactions

207 Tool calls (MCP)

10 Command-line commands

3 Web / HTTP endpoints

Also in this family: live socket connections (WebSocket), query APIs (GraphQL, gRPC), pages & routes.

Send & forget ASYNCHRONOUS

The mail slot and the alarm clock — something arrives; the system reacts; nobody's on the line.

A message, event, or timer arrives and the system does work in response, but there's no caller waiting for a reply. This is how systems talk to *each other* and to schedules.

228 Messages / streams

1 Scheduled jobs

Also in this family: published events & topics, incoming webhooks.

Reacts on its own PASSIVE

The motion-sensor lights — no one calls them; the environment trips them.

Nothing external "calls" these. The system's own runtime or operating environment triggers them — the app starting up or shutting down, a file appearing in a watched folder, a hardware signal. Easy to overlook, which is exactly why surfacing them earns its keep.

In this family: startup & shutdown lifecycle, file watchers, hardware / timer signals.

The mix *is* the system's character: a heavy tool-call surface says "built for AI agents"; a heavy message surface says "event-driven backend"; a heavy pages-and-clicks surface says "user-facing web app." A reviewer reads the shape before reading a single row.

THE FULL VOCABULARY

Every kind of entry point

The three families hold a **fixed, complete set** of kinds — Klauro recognizes them all, even ones a given system won't have. Here's the whole vocabulary, so nothing you might meet is a surprise. Any one deployable uses a handful; the rest simply read as zero. You don't need a distinct treatment for all of these on day one — but you should know the full list exists before deciding how to name and mark them.

Request & wait		SYNCHRONOUS
Web / HTTP endpoint	A URL the system answers, with a verb (GET, POST...).	
API operation	A declared API call, often one of many under a shared spec.	
Web route / page	A page or route a browser loads directly.	
GraphQL query	A single GraphQL field or mutation.	
gRPC / RPC method	A remote procedure call — a method, not a web path.	
Live socket (WebSocket)	A persistent two-way connection held open.	
Command-line command	A subcommand run in a terminal.	
Desktop app command	A command a desktop UI calls into its own backend.	
In-app message (IPC)	A call from a desktop UI to its background process.	

UI interaction	A button or form handler in a web app. (See the note below.)
-----------------------	--

Send & forget	ASYNCHRONOUS
Message / queue	An item pulled from a queue and handled.
Pub/sub topic	A subscription to a broadcast topic.
Event stream	A handler on a continuous stream of records.
Published event	An event the system emits or reacts to internally.
Incoming webhook	An HTTP callback another service posts to you.
Scheduled job	A timed, recurring trigger (a cron schedule).
Pipeline task / run	A data-pipeline step, or a whole pipeline run.

Reacts on its own	PASSIVE
Startup / shutdown	Runs as the app boots or stops.
Lifecycle hook	Runs at a defined moment in the app's life.
File watcher	Fires when a file appears or changes.
Hardware interrupt	A timer or hardware signal (embedded systems).
Driver hook	A kernel or driver callback (embedded systems).

Specialized	DATA / ML
ML training entry	The start of a model training loop. Rare outside ML systems.
Notebook cell	An ordered cell in a data notebook. Rare outside data work.

One kind lives in two families

An *event* can be either: a UI click is request-and-wait; a published or consumed event is send-and-forget. Context decides which family it belongs to — worth knowing if you build

anything that keys off the family.

THE KEY RELATIONSHIP

Every entry point opens into a flow

An entry point is not the end of the story — it's the *beginning* of one. The moment an entry point fires, the system runs a **flow**: the ordered sequence of steps that entry point sets in motion. There is one flow behind every entry point, and it's the answer to the question a list of entry points immediately provokes — "*okay, but what does it actually do?*"

So the entry point and its flow are a pair: the entry point is **how you get in**, the flow is **what happens next**. The natural interaction is to move from one to the other — select an entry point, and drop into the flow it triggers.

ENTRY POINT

POST /v1/analyze

A web request arrives, protected by an access token.



THE FLOW IT OPENS

Analyze a codebase

Validate the request → load the workspace
→ run the analysis → save the result.

Each flow carries its own detail, ready for a deeper view: what it **takes in**, the **data it touches**, the **outside services it calls** along the way, what it **returns**, and where it **ends**. That richness lives on the flow, not the entry point — so however this is presented, leave a door open from each entry point into its flow rather than trying to show everything in one place.

Why this matters for the design

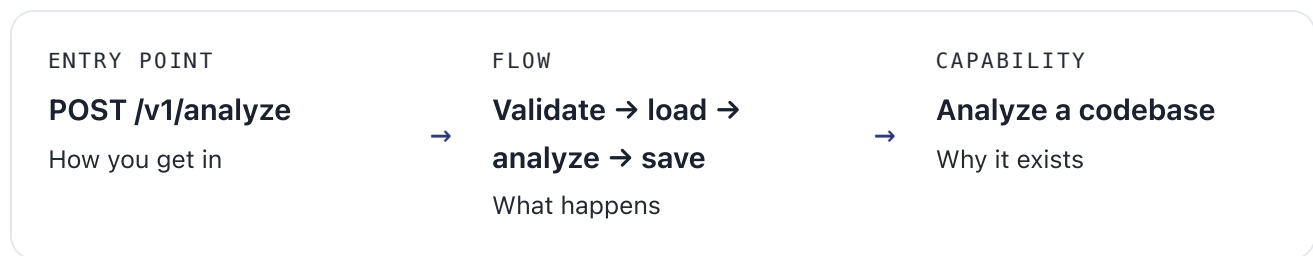
It means an entry-point view is a *starting surface*, not a terminal one. The reader lands here to get oriented — the counts, the kinds, what's protected — then follows the thread inward to a specific flow. Designing the step *from* entry point *into* flow is as important as the catalog itself.

ONE RUNG HIGHER

Most entry points ladder up to a capability — but not cleanly

There's one more link, and it's the subtlest one in the brief. If a flow is *what happens* when

an entry point fires, a **capability** is *why it exists* — the business thing the system does. Klauro names these too (in this analysis: "Analyze a codebase," "Detect patterns," "Trace and monitor"). So the ladder runs entry point → flow → capability:



But that last link is **not one-to-one, and not universal** — this is the part to design carefully around, because a naive "each entry point belongs to one capability" will misrepresent the system. Three things are true:

01 It's many-to-many, with a role each way

One capability is realized by many flows; one flow can serve several capabilities. And a flow carries a *role* for each tie — it's the **primary** flow that realizes one capability, and merely **supporting** for others.

02 Cross-cutting entry points belong to (almost) everything

Authentication is the clearest case — the auth flow isn't the point of any single capability; it *supports* nearly all of them. Same for logging, validation, rate-limiting. These recur across many capabilities by design, so an entry point can legitimately map to a long list, not one.

03 Some entry points serve no capability at all

Build and deploy scripts, health checks, internal plumbing — real entry points whose flow has a role of **infrastructure**. They ladder up to nothing on the business side, and that's correct, not a gap.

With that understood, capabilities unlock a **second way to organize the same entry points** — by *purpose* instead of by *mechanism*. Rather than "here are 207 tool calls," it becomes: "the *Analyze a codebase* capability can be triggered five ways." That's usually the question a person actually arrives with — "*how do I make it do X?*" — and it surfaces health signals a flat list can't: a capability with **no** entry point (built but untriggerable), or a capability reachable only **one** way (a single point of entry).

What this means for a by-capability view

Design it to hold three shapes at once: an entry point that belongs to **one** capability, one that belongs to **many** (auth, health), and one that belongs to **none** (infrastructure). A layout that assumes a tidy one-to-one will break on the first cross-cutting entry point — and every real system has them.

What we know about a single entry point

For each individual entry point, this is the information available to show. Some of it is for anyone; some is for a technical reader who wants to jump into the code. You can choose what to reveal, to whom, and when.

WHAT IT IS	WHAT IT TELLS THE READER
Name & description name · description	A human-readable label and a plain sentence on what this entry point is for. The primary thing a person reads.
Its address trigger	How you actually reach it: a web address and verb (<code>POST /v1/analyze</code>), a tool name, a command, a schedule (" <i>every night at 2am</i> "), or the name of an event it listens for.
Whether it's protected security	Does reaching it require a login or a permission, or is it open to anyone — and who's allowed. The heart of the security lens.
What it expects input	The shape of what you send in, and whether the system checks it before acting.
What it returns output	The shape of the response and its status — for request-and-wait entry points.
The flow it opens flow	The behavior it triggers — the way through to "what actually happens." (See the section above.)
Where it lives in code handler · file · line	The exact file and line the entry point leads to. For a developer to jump straight there; hide it from non-technical views.
Kind & how we found it type · source	Which channel it is and how it was detected. Mostly useful for grouping, not for reading.

What those fields actually contain

Several fields above are abstract until you see them filled in — and the important thing is that **their content changes with the kind of entry point**. A web endpoint's inputs are body fields with types; a command's are flags; a scheduled job has none. Here's the real content to expect, so you know what any container for these has to hold. (Examples use Klauro's own API.)

What it expects the “input”

Usually a list of named inputs, each with a type and a required-or-optional marker. For some kinds it's command flags or a message payload instead — and for others, nothing at all. A field's type can be a plain value (text, number) or a named type with its own definition — the same range as returns, below. Your container needs to hold a short list of “name · type · required?”, read well when the list is empty, and let a named type link deeper.

Web endpoint · POST /v1/analyze

project_path	text	REQUIRED
snapshot	file bundle	REQUIRED
branch	text	OPTIONAL

Command · klauro analyze

path	text	REQUIRED
--with-ai	on / off	OPTIONAL
--budget	number	OPTIONAL

Scheduled job · nightly refresh

Nothing. It takes no input — it just fires on its timer. Many async and passive entry points look like this; the empty state is a real, common case.

What it returns the “output”

This one has the most range, so it's worth spelling out. A return value can be a **named type** — a class or interface that has its own definition elsewhere, which means the return can be a *link deeper*, not just a label. Or a loose object with inline fields. Or a plain value. Or a list. Or one of several things. Or nothing at all. On top of that, request-and-wait entry points also carry **outcome codes** (the success and the failures). Your container has to hold all of these gracefully — and treat a named type as something the reader can follow, the way they follow a flow.

The forms a return can take

Web endpoint · POST /v1/analyze

Named type a class or interface — e.g. **AnalysisResult** — with its own definition to open

Object shape inline fields — { stdout: text, code: number }

Plain value true / false, a number, a piece of text

A list many of any of the above

One of several a result or nothing; one of a few named types

Nothing void — it returns no value (common for send-and-forget)

Returns an **AnalysisResult** — a named type you can open

OUTCOME CODES

202 Accepted 401 Not signed in
400 Bad request

A message handler, by contrast, returns **nothing** — its result is the work it does. You see that by following its flow.

Whether it's protected the “security”

This is more layered than a lock icon. First the plain state — **open to anyone** or **protected**. Then, when protected, four separate things, and an entry point may carry any mix: **how you prove who you are** (the method — an access token / JWT, a login session, an API key, a sign-in provider, a client certificate); **who's allowed** (the roles — the RBAC layer); **what permissions, scopes, or claims** a caller must carry; and **any limits** (a rate cap). Your container needs an at-a-glance state, plus a home for each layer when it's present.

GET /health

● Open to anyone

Public — no login needed. Anyone who can reach the system can call it.

POST /v1/analyze

● Protected

Method access token (JWT)

Role workspace member

Must have the claim “owns this project”

Limit capped request rate

Its address the “trigger”

The one other field whose content shifts entirely with the kind — there's no single format. It's a verb and a web path, or a tool name, or a full command line, or a schedule in words, or the name of an event. Whatever holds it can't assume a fixed shape.

Web endpoint POST /v1/analyze

Scheduled every day at 02:00 UTC

Tool call analyze_codebase

Command klauro analyze ./project --with-ai

Message on event "analysis.completed"

File watcher when a file lands in
/uploads

Security may deserve its own view — worth settling early

Security is the one dimension rich and cross-cutting enough that it might not belong *inside* the entry-point view at all. Beyond the per-entry summary above, Klauro computes it at the system level: the **boundaries** a request crosses (an "authentication boundary," enforced by middleware), whether each is **actually enforced or only assumed**, and — the part a reviewer cares most about — a list of **sensitive operations left unprotected**. That's a whole story of its own. A reasonable shape: a dedicated security view that aggregates boundaries, enforcement, roles, and the unprotected-surface risk list, while the entry-point view keeps a compact per-entry summary that links into it. Whether security lives here, there, or both is a real fork to decide up front.

THE LIVE LAYER

What we know once it's running

Everything so far is read from the code without running it — static. But an entry point also has a life *at runtime*, and Klauro can capture it: how often it's actually called, how often it fails, and how fast it responds. This is a different kind of data — streamed from the running system — and it turns the catalog from a map into a **health view**. It's the one part of the picture that's about the present moment, not the code.

For a single entry point, the runtime picture looks like this:

TRAFFIC

12.4k

calls in the last day

ERRORS

2.1%

share of calls that
failed

TYPICAL SPEED

40ms

median response
(p50)

WORST CASE

210ms

slow tail (p95 /
p99)

Small as it is, that changes what the view can answer:

- **Live vs. dormant** — which entry points are actually used, and which are built but never called (dead weight, or a security surface no one's watching).
- **The hot paths** — the handful of entry points carrying most of the traffic, worth the most care.
- **The trouble spots** — high error rate or a slow tail. the ones to look at first.

It's only there when the system is reporting

This data exists only for a deployment that's *running and instrumented* — a system Klauro has analyzed but that isn't sending telemetry shows none of it. So treat the live layer as **present sometimes, absent others**: the static picture always stands on its own, and the runtime numbers enrich it when they're there. Design the entry-point row to look complete without them, and better with them.

MORE WAYS TO CUT IT

Other dimensions you could organize by

The three families and the capability view are the two big organizing ideas, but the same entry points carry several other dimensions. Any one could be a grouping, a filter, a sort, or a color. These are the ones grounded in real data — a few need data we're still filling in, marked accordingly.

Protected vs. public

Behind a login versus open to anyone — the security lens. The most compelling cut for a reviewer.

External vs. internal

Reachable from outside the system, or only by sibling deployables. This is the true "attack surface" — what the outside world can actually touch.

Used vs. dormant **NEEDS DATA**

How often each entry point is actually hit, from live traffic — the live doors, the dead ones, the error-prone ones. Turns a static catalog into a health view.

Registration surface

Klauro already pre-groups entry points that are wired up the same way (e.g. "the tool-call surface") — a ready-made grouping you can lean on.

Resource / domain

The noun a set of endpoints clusters around — everything under "projects," everything under "workspaces." A familiar way to browse an API.

First-party vs. framework

Did a developer write this, or is it a framework default (e.g.

First-party vs. framework

Did a developer write this, or is it a framework default (a health check, a lifecycle hook)? Separates the signal from the boilerplate.

Reach / blast radius

How much of the system sits behind an entry point — which doors open onto the most.

The mirror image: exit points

Entry points are how the system is *reached*. The same analysis also finds **exit points** — how the deployable reaches *out*: to databases, other services, message buses, caches. In this analysis there are **1,227** of them (802 database, 331 libraries/SDKs, 58 outbound APIs, 20 messages). Ways-in and ways-out together are the *complete* surface of contact between this system and everything around it. Whether that belongs on the same view or a neighbouring one is your call — but it's worth knowing the other half exists.

DESIGN AROUND THESE

The shape of the real data

Five facts about how this data actually looks — the ones most likely to make a naive treatment fall apart.

01 The count ranges wildly

A small service might have a dozen entry points; a large one, a couple of thousand. This analysis has **737** (excluding tests). The design has to feel right at both a dozen and two thousand — which usually means summarize-first, with search and grouping to reach any single one, not a scroll of everything.

02 The distribution is extremely lopsided

A couple of channels hold hundreds of entry points; many hold one or two. If every category gets equal visual weight the result feels wrong — the big surfaces need room to breathe and the long tail needs a compact treatment. A one-item group shouldn't claim the same real estate as a two-hundred-item one.

03 The names aren't always friendly

Roughly a third of entry points are labeled with raw code names — `main` , `handleRequest` , `do_thing` — not clean phrases like "Analyze a codebase." Your treatment of the name has to hold a tidy sentence *and* a terse code token without looking broken, and probably show the address right alongside so a cryptic name isn't the only

handle.

04 Detail varies by kind

A web endpoint naturally carries a rich contract — inputs, outputs, who's allowed. A lifecycle hook carries almost nothing but a name. Whatever you design has to look intentional both when an entry point is detail-rich and when it's a bare label. Plan the sparse case first.

05 The character shifts hard per deployable

Switch from the web app to the API server and the whole complexion changes — pages-and-clicks become tool-calls-and-messages. This isn't one flat list with a filter; it's a context the viewer stands inside. Switching deployables should feel like changing rooms, not re-sorting a table.

Don't forget the in-between states

The happy path — a rich, fully-populated entry point — is the easy case to design. The ones that decide whether this feels solid: a deployable with **no entry points found**; an analysis **still running**; an entry point with only a name and address and **nothing else**; **no telemetry** because the system isn't reporting. Each of these is common, not exceptional. A design that only looks right when everything's present will look broken most of the time.

THE WHOLE PICTURE

How it all fits together

Every piece named in this brief nests inside the next. Seeing the whole structure at once makes the rest easier to hold — and shows where the viewer is standing when they look at entry points.

System — the whole product or codebase

Deployable — one independently running piece (API server, web app, worker)

Entry point **YOU ARE HERE** — a way in, grouped by family & channel

Flow — what happens when it fires

Capability — the purpose it serves (many-to-many)

Two things describe every entry point but aren't levels — they cut across:

Security — who may enter

Telemetry — how it behaves live

And what a person does from here. Worth deciding whether this is a reference to read or a tool to operate — the data supports either. From a single entry point, the natural moves:

- **Get oriented** — read the counts, the families, what's protected, what's live.
- **Follow it inward** — open the flow it triggers, to see what actually happens.
- **Jump to the code** — for a technical viewer, straight to the file and line.
- **See its purpose** — the capability it serves, and what else serves that capability.
- **Cross to security** — from a protected entry point into the fuller security picture.
- **Slice the whole set** — group, filter, or sort by any of the dimensions above.

REFERENCE

Engineer-speak ↔ plain English

If you end up talking with the engineering team, these are the terms they'll use for the things named plainly above. You never need to put these words in front of a user — it's just a decoder ring.

THEY SAY	IT MEANS
entry point	A way into the running system (the subject of this brief)
deployable	One independently running piece of software
modality	The sync / async / passive split — the three families
flow	The sequence of steps one entry point sets in motion
capability	The business purpose a flow serves — the "why"
exit point	A way the system reaches out (database, API, queue) — the mirror of an entry point
behavior surface	A pre-grouped set of entry points wired up the same way

trigger	The address — how an entry point is reached
handler	The code an entry point leads to
MCP tool	A command exposed for AI agents to call
HTTP / REST route	A web address the system answers
message / event / topic	Something that arrives to be reacted to
schedule / cron	A timed, recurring trigger
lifecycle	App startup / shutdown handlers

Klauro · Entry-point analysis · Counts from a live CAS analysis, test entry points excluded · Plain-language design handoff